Stem Agent: Self-Evolving Reasoning Graph System

1. Overview

This task explores the concept of a “stem agent” an AI system that begins with minimal structure and evolves into a specialized reasoning agent through iterative refinement.

Instead of manually designing task-specific pipelines the system dynamically discovers and optimizes its own reasoning structure using:

- Graph-based execution
- Evolutionary search (mutation + selection)
- Multi-factor evaluation

Inspired by biological stem cells, which differentiate based on signals, this system starts with a diverse population of reasoning graphs and evolves them toward task-specific specialization.

2. Problem Scope & Motivation

The system is designed for multi-step reasoning tasks, such as:

- Security Analysis
- Test Planning
- Structured Research Tasks

These tasks were chosen because:

- They require clear decomposition into steps
- There is no single correct pipeline
- They expose weaknesses in naïve LLM usage

3. System Architecture

The system consists of three core components:

3.1 Reasoning Graph (Agent Representation)

Each agent is represented as a directed graph:

- **Nodes** = reasoning steps, **Edges** = execution order

Each node contains:

- A purpose (e.g., “Threat Identification”)
- A prompt guiding the LLM

3.2 Graph Execution Engine

Execution is handled by GraphCompiler.

Key design decision:

Each node MUST only perform its assigned step.

To enforce this:

- Each node receives only the previous step output
- It is explicitly stated: Do NOT solve the full task; only perform this step.

This prevents a common failure mode where LLM collapses the entire reasoning into one step.

3.3 Evolution Engine

The system evolves agents using a loop:

Step 1: Initialization

Instead of identical agents, a diverse population is created:

- Base structured agent
- Randomly mutated agent
- Creative prompt variant
- Minimal 2-step agent

This improves exploration and avoids early stagnation.

Step 2: Execution

Each agent is executed:

- Node-by-node
- Passing outputs sequentially

Outputs are stored per node.

Step 3: Evaluation (Fitness Function)

Each agent is scored using multiple signals:

Component	Purpose
LLM Judge	Output quality
Structure Score	Logical flow
Diversity Bonus	Avoid repetition
Reflection Score	Self-improvement signal
Empty Penalty	Penalize weak steps
Noise	Avoid local minima

This multi-objective fitness ensures:

- Structured reasoning is rewarded
- Incomplete pipelines are penalized

Step 4: Mutation & Selection

- Top agents are retained
- New agents are created via mutation
- Each agent stores its parent lineage

This enables tracking how reasoning evolves.

4. Evaluation Method

4.1 Before vs After (Structural Evolution)

Initial Agent:

Understand → Reason → Answer

Final Agent:

Threat Identification → Vulnerability Analysis → Risk Assessment → Mitigation Planning

Improvement:

- More domain-specific
- More interpretable
- Better decomposition

4.2 Node-Level Output Quality

Before:

- Only 1 node produced useful output
- Others were empty

After enforcing constraints:

- Every node produces meaningful output
- Clear step-by-step reasoning emerges

4.3 Fitness Trend

Across generations:

- Weak agents are eliminated
- Structured pipelines dominate
- Fitness stabilizes at higher values

4.4 Measurable Signal

- Reduction in empty outputs

- Increase in node coverage
- Higher evaluation scores from LLM judge

5. Key Experiments

Experiment 1: Identical vs Diverse Initialization

Observation: Identical agents led to no evolution

Fix: Introduced diverse starting population

Result:

- More varied reasoning structures
- Better final agents

Experiment 2: Step Responsibility Enforcement

Problem: LLM ignored pipeline and answered directly

Fix: Forced each node to:

- Use previous output
- Solve only its step

Result:

- True multi-step reasoning

Experiment 3: Weak Fitness Signal

Problem: All agents scored similarly

Fix: Added penalties and structure scoring

Result:

- Clear selection pressure
- Meaningful evolution

6. What Surprised Me

- LLMs naturally **collapse multi-step reasoning** unless constrained
- Diversity in initial population has a **huge impact**
- Reflection signals even simple ones improve selection
- Structure matters as much as content

7. What Failed

- Identical initialization → no evolution

- Weak mutation → repetitive agents
- No penalties → bad agents survive
- No step constraints → pipeline ignored

These failures shaped the final design.

8. What I Would Do With More Time

- Add tool usage (search, execution)
- Introduce parallel reasoning graphs
- Improve mutation via learned transformations
- Replace LLM judge with trained evaluator